

BharatPulse — Build Plan & AI Coding Brief

Paste this entire document into Claude Code, Cursor, v0, Replit, ChatGPT, or any AI coding assistant to start building. It's written as a self-contained spec — the AI should be able to scaffold the whole project from this alone, asking you only for API keys.

1. What we're building

BharatPulse is a web app (not mobile — responsive site only, PWA-ready) that aggregates Indian news into "Topic Hubs." Each hub clusters coverage of one real-world event from three lanes:

- 1. **Mainstream** — national English/Hindi dailies (official/institutional framing)
- 2. **Grassroots** — independent/investigative outlets and creators (ground reporting)
- 3. **Public Discourse** — Reddit threads on the same topic (citizen reaction)

No human editors. AI does topic clustering. Everything is pulled from free, legal, open pipes: RSS, YouTube RSS, Reddit RSS/API, OEmbed. No scraping behind paywalls, no scraping social platforms directly.

V1 scope is a website only. No native app, no push notifications, no accounts required to browse.

2. Tech stack (all free-tier to start)

Layer	Choice	Why
Frontend	Next.js 14+ (App Router), TypeScript, Tailwind CSS	SSR for SEO on topic hub pages, huge ecosystem, deploys free on Vercel
Hosting	Vercel (free Hobby tier)	Zero-config Next.js deploys, generous free tier
Database	Supabase (Postgres, free tier)	Free Postgres + built-in vector extension

Layer	Choice	Why
		(pgvector) for clustering + auth if needed later
Scheduled ingestion	GitHub Actions (cron)	Free scheduled jobs, no server to maintain, runs the RSS-pull + clustering script every N minutes
AI inference (clustering + summarization)	Start with a free/cheap tier: Groq (free tier, fast Llama models) or Google Gemini free tier. Fallback: OpenAI/Anthropic API on a small budget if free tiers aren't enough	Needed for embedding generation (clustering) and 2-line summaries
Embeddings	Any free embedding API (Gemini embedding, or open-source <code>all-MiniLM-L6-v2</code> run inside the GitHub Action itself via <code>sentence-transformers</code> — fully free, no API needed)	Clustering doesn't need a huge model — a lightweight local embedding model is enough and costs nothing
RSS parsing	<code>rss-parser</code> (Node) or <code>feedparser</code> (Python)	Standard, free
Reddit data	Reddit's public JSON/RSS endpoints (<code>reddit.com/r/india/.rss</code>) — no API key needed for read-only	Free, no auth
YouTube data	YouTube RSS (<code>youtube.com/feeds/videos.xml?channel_id=X</code>)	Free, no API key needed for basic feed (only need API key for search, not for known channels)

Total infra cost to launch: \$0. You only start paying if you outgrow Supabase's 500MB free DB or need heavier AI inference volume — cross that bridge later.

3. Data model (Supabase / Postgres)



sql

-- Sources: every RSS/YouTube/Reddit feed we ingest

```
create table sources (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  lane text not null check (lane in ('mainstream', 'grassroots', 'discourse')),  
  type text not null check (type in ('rss', 'youtube', 'reddit')),  
  feed_url text not null,  
  language text not null default 'en',  
  region text, -- e.g. 'national', 'tamil-nadu', 'west-bengal'  
  is_active boolean default true,  
  created_at timestampz default now()  
);
```

-- Raw items pulled from sources, before clustering

```
create table raw_items (  
  id uuid primary key default gen_random_uuid(),  
  source_id uuid references sources(id),  
  title text not null,  
  url text not null unique,  
  published_at timestampz,  
  raw_summary text, -- RSS description/snippet, NOT full scraped article  
  embedding vector(384), -- pgvector, matches MiniLM output size  
  cluster_id uuid, -- nullable until clustered  
  fetched_at timestampz default now()  
);
```

-- Topic hubs: the clustered "same event" groups

```
create table topic_hubs (  
  id uuid primary key default gen_random_uuid(),  
  title text not null, -- AI-generated hub title  
  ai_summary text, -- short neutral summary, generated with caution (see Section 6)  
  first_seen_at timestampz default now(),  
  last_updated_at timestampz default now(),  
  item_count int default 0  
);
```

```
alter table raw_items
```

```
add constraint fk_cluster foreign key (cluster_id) references topic_hubs(id);
```

Enable `pgvector` extension in Supabase for the embedding column and similarity search (cosine distance) used in clustering.

4. Core pipeline (runs every 15–30 min via GitHub Actions)

1. **Fetch** — loop through all active `sources`, pull new items since last run (dedupe by URL).
2. **Embed** — generate a vector embedding for each new item's title + snippet using a local lightweight embedding model (no API cost).
3. **Cluster** — compare new item's embedding against existing `topic_hubs` from the last 48–72 hours using cosine similarity. If similarity > threshold (start at 0.80, tune later), attach to that hub. Otherwise, create a new hub.
4. **Summarize (optional, cautious)** — only after a hub has items from at least 2 different lanes, generate a short neutral summary describing what's being reported, not adjudicating truth. See Section 6 for the exact prompt guardrails.
5. **Publish** — hub becomes visible on the site once it has ≥ 2 sources.

Write this as a single Node.js or Python script, triggered by a GitHub Actions cron job (e.g. `* / 20 * * * *`).

5. Site structure (pages)

- `/` — Home: live feed of active Topic Hubs, most recently updated first. Each card shows: hub title, item count per lane (e.g. "12 mainstream · 4 grassroots · 38 discourse threads"), time since first seen.
- `/hub/[id]` — Topic Hub page: three-column (or tabbed on mobile-width) layout — Mainstream / Grassroots / Discourse. Each item: source name, headline, timestamp, link out to original. AI summary at top, clearly labeled "AI-generated overview" with a disclaimer.
- `/sources` — Transparency page: full list of every source ingested, its lane, and language/region. This matters for trust — show your work.
- `/about` — What BharatPulse is, how clustering works, explicit statement that it does not editorialize or fact-check, links to independent fact-checkers (Alt News, BOOM, etc.) for verification.

No login wall for V1. No comments section (avoid moderation burden and legal exposure in V1 — this can come in V2 if at all).

6. AI summary guardrails (important — read before implementing)

- The AI summary must **only describe what is being reported and by whom**, never adjudicate what's true. Prompt template:
"Summarize in 2 neutral sentences what is being reported across these headlines, without taking a position on which account is correct. Attribute claims to 'reports say' or name the lane (mainstream/grassroots/discourse), not as established fact."
- Never let the summary model output a single merged "fact" from conflicting sources — if sources disagree, say they disagree.
- Always show the disclaimer: *"This summary is AI-generated from public headlines and may not capture full context. Read the original sources."*
- Do not auto-summarize hubs related to active communal violence, ongoing legal cases, or election-day results in V1 — flag these for a "sources only, no AI summary" mode until you've stress-tested the summarizer on lower-stakes topics first.

7. Build phases (give the AI one phase at a time if it struggles with the whole spec at once)

Phase 1 — Skeleton (week 1)

- Next.js project scaffolded, Tailwind configured, Supabase project connected
- `sources` table populated manually with ~20 starter feeds (mix of all 3 lanes)
- One script that fetches RSS and inserts into `raw_items`, run manually (not yet scheduled)

Phase 2 — Clustering (week 2)

- Add embedding generation + pgvector similarity search
- Auto-create/attach `topic_hubs`
- Basic home page listing hubs, no styling polish yet

Phase 3 — Hub pages + AI summary (week 3)

- Build `/hub/[id]` three-lane view
- Add cautious AI summarization with guardrail prompt from Section 6
- Add `/sources` and `/about` pages

Phase 4 — Automation + polish (week 4)

- Move ingestion script to GitHub Actions cron

- Add more sources (aim for 50–80 across all lanes/languages from the source list)
- Responsive design pass, loading states, empty states
- Deploy to Vercel, connect custom domain if desired

Phase 5 — Post-launch (ongoing)

- Add regional-language source coverage
- Add fact-checker lane (4th column) linking Alt News/BOOM/Newschecker coverage of the same topic
- Consider a public API for the clustered dataset (open-source ethos)

8. Non-goals for V1 (explicitly tell the AI NOT to build these yet)

- No native mobile app
- No user accounts/login
- No comments or user-generated discussion
- No push notifications
- No scraping of paywalled sites (The Hindu, Mint, Business Standard, Newslaundry, The Caravan, The Ken, ET Prime) — link out to them, don't pull full text
- No direct PTI/ANI ingestion (licensing issue — see prior research)
- No "bias score" or political-lean labeling of sources in V1 — that's a legally and editorially sensitive feature to get right; ship the three-lane structure first and revisit

9. Starter source list to seed the | | |---------| | sources | |---------| table

Use the free-tier list already compiled: Indian Express, Times of India, NDTV, Deccan Herald, Dainik Jagran, Dainik Bhaskar, PIB (mainstream lane); The Wire, Scroll.in, The News Minute, Newslaundry (free portion), Article 14, The Mooknayak, Gaon Connection, PARI (grassroots lane); r/india, r/IndiaSpeaks, r/indiatech RSS (discourse lane). Expand from there using the full regional list once the pipeline works end to end.

10. Instructions to the AI building this

Build this incrementally, phase by phase as listed in Section 7. After each phase, show me the running app before moving to the next. Ask me for API keys/credentials only when a phase

actually needs them (Supabase URL + anon key for Phase 1, an embedding provider key only if not using a local model). Keep the codebase in TypeScript for the Next.js app. Comment the clustering logic clearly since the similarity threshold will need tuning based on real data. Do not implement anything listed in Section 8 as a non-goal unless I explicitly ask.